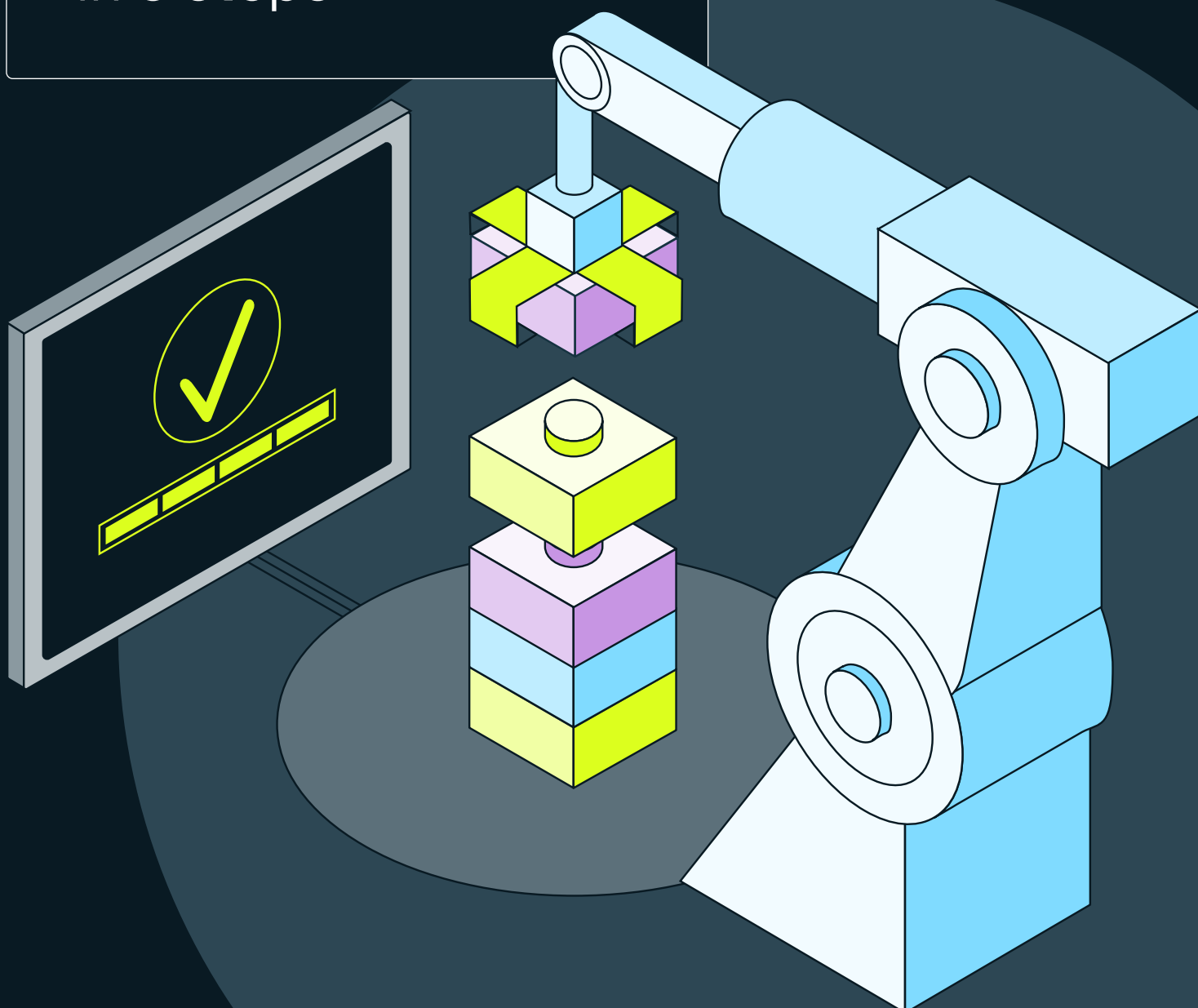




Written by Talon Miller
Principal Technical Marketer

● WHITEPAPER

Build an AI app in 5 steps



Step 1: Add fast LLM memory with a Redis vector database	03
Step 2: Speed up responses with semantic caching	05
Step 3: Add agentic memory for smarter, adaptive agents	06
Step 4: Keep your app safe with rate limiting	07
Step 5: Speed up ML predictions with Redis feature store	08
Final thoughts & what to build next	09

AI-powered apps are reshaping how we interact with technology—powering smarter chatbots, personalized recommendations, advanced search, and more. But turning ideas into production-ready AI experiences isn't simple. Devs face challenges like high latency, inaccurate results, scaling limitations, and complex infrastructure.

That's where Redis comes in.

Redis is a fast, flexible, and scalable data platform built for real-time AI workloads. From vector

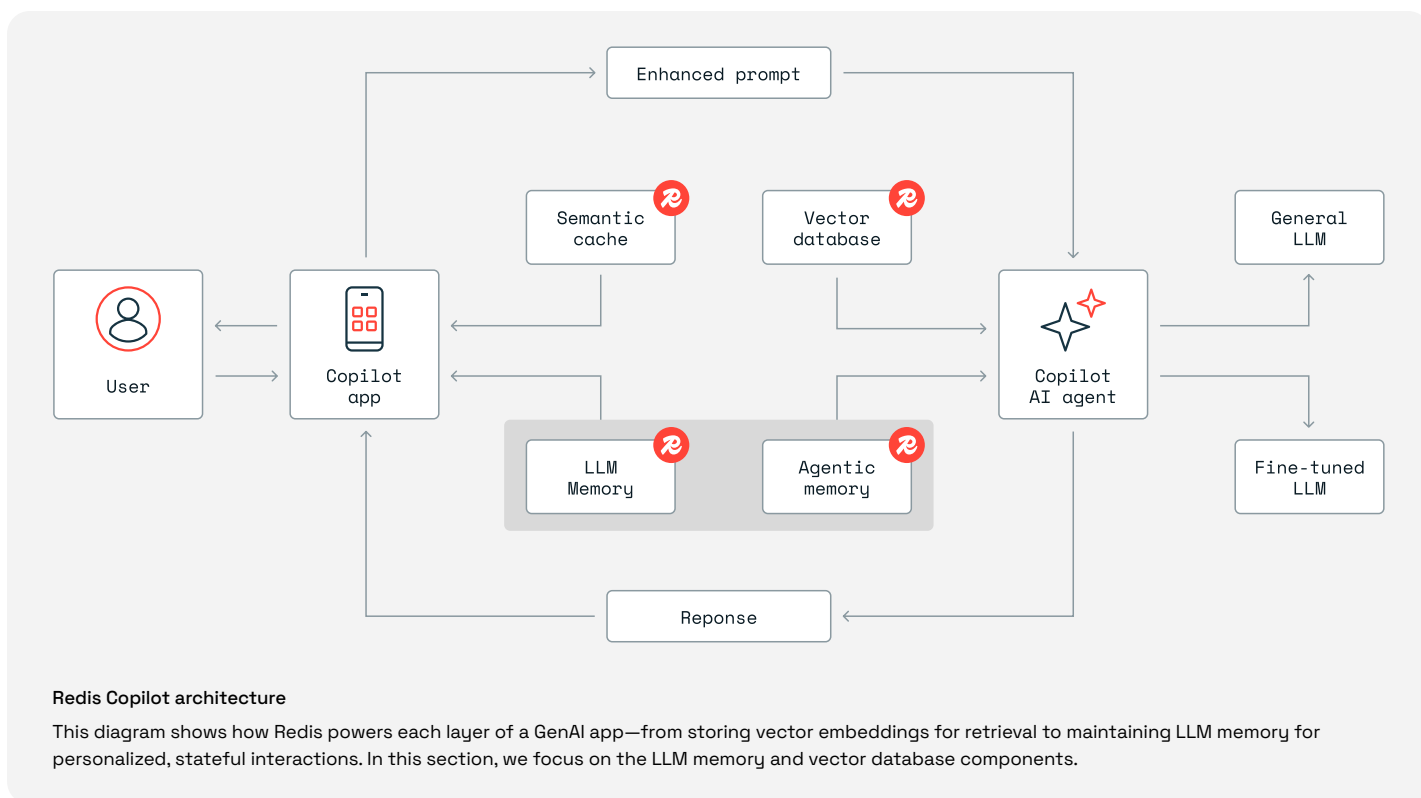
similarity search to semantic caching and dynamic rate limiting, Redis provides the tools you need to build AI apps, without overcomplicating your stack.

In this guide, we'll walk through five key steps to building modern AI apps using Redis. Each step includes what to implement, why it matters, and how Redis helps at every layer of the stack. Whether you're building your first retrieval-augmented generation (RAG) app or scaling intelligent agents in production, this guide offers a practical blueprint to get you there faster.

Step 1: Add fast LLM memory with a Redis vector database

To make LLM-powered apps more useful and personalized, they need memory: the ability to remember what a user has said before, understand ongoing context, and adjust

responses accordingly. In this first step, we'll look at how Redis helps you build that memory layer fast by combining native vector similarity search with real-time data handling.



Redis Copilot, the Redis AI assistant, was built with this exact goal in mind. One of the first features implemented to build Copilot was LLM Memory, so the assistant could recall past instructions like “always include a link” or maintain a persona like “Redis expert.” Without memory, users would have to repeat themselves every time. With Redis, these preferences can be stored once and persist across the entire conversation.

To implement LLM memory, the process starts with chunking. Whether you’re working with documentation or user input, the text is broken down into semantically meaningful chunks—usually around a paragraph in length. This ensures each chunk represents a distinct idea or concept, which improves the relevance of retrieval. Redis Copilot uses LangChain to do this preprocessing on Redis documentation, preparing the data for vectorization.

Once chunked, each piece of text is passed through an embedding model to generate vector representations. These embeddings are then stored in the Redis vector database. This setup enables fast, scalable similarity search—critical for real-time apps. Redis Vector Library (RedisVL) makes it simple to handle this process by abstracting the complexity behind intuitive APIs.

Retrieving relevant memory is handled via k-nearest neighbors (KNN) vector search. This allows the app to compare a user’s current input with previous chunks or instructions stored in Redis. But instead of just returning the top K results, Redis Copilot takes an additional step to rerank those results using RedisVL.

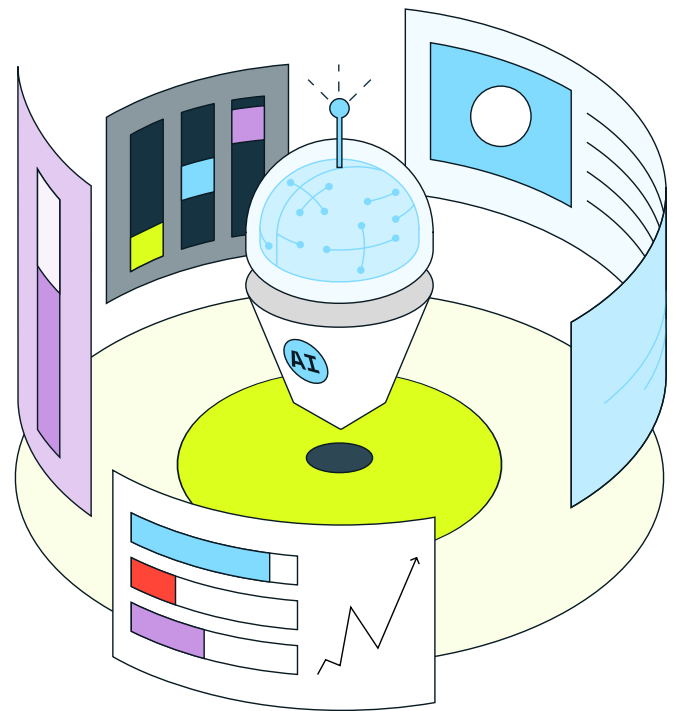
This reranking ensures that only the most relevant context is passed to the LLM, which improves the quality of responses while keeping latency low.

Finally, Redis is used to store and retrieve personalized memory. For example, if a user instructs the LLM to behave like a financial advisor or to avoid using previously suggested answers, those preferences are stored and retrieved dynamically as part of the session context. This makes the experience feel much more adaptive and intelligent, without forcing the user to repeat themselves.

All of this is shown in action in [Minipilot](#)—a lightweight version of Redis Copilot. It demonstrates how to combine RAG, vector search, and memory handling to deliver smarter GenAI experiences.

[Watch episode 1 →](#)

LLM memory and vector database



Step 2: Speed up responses with semantic caching

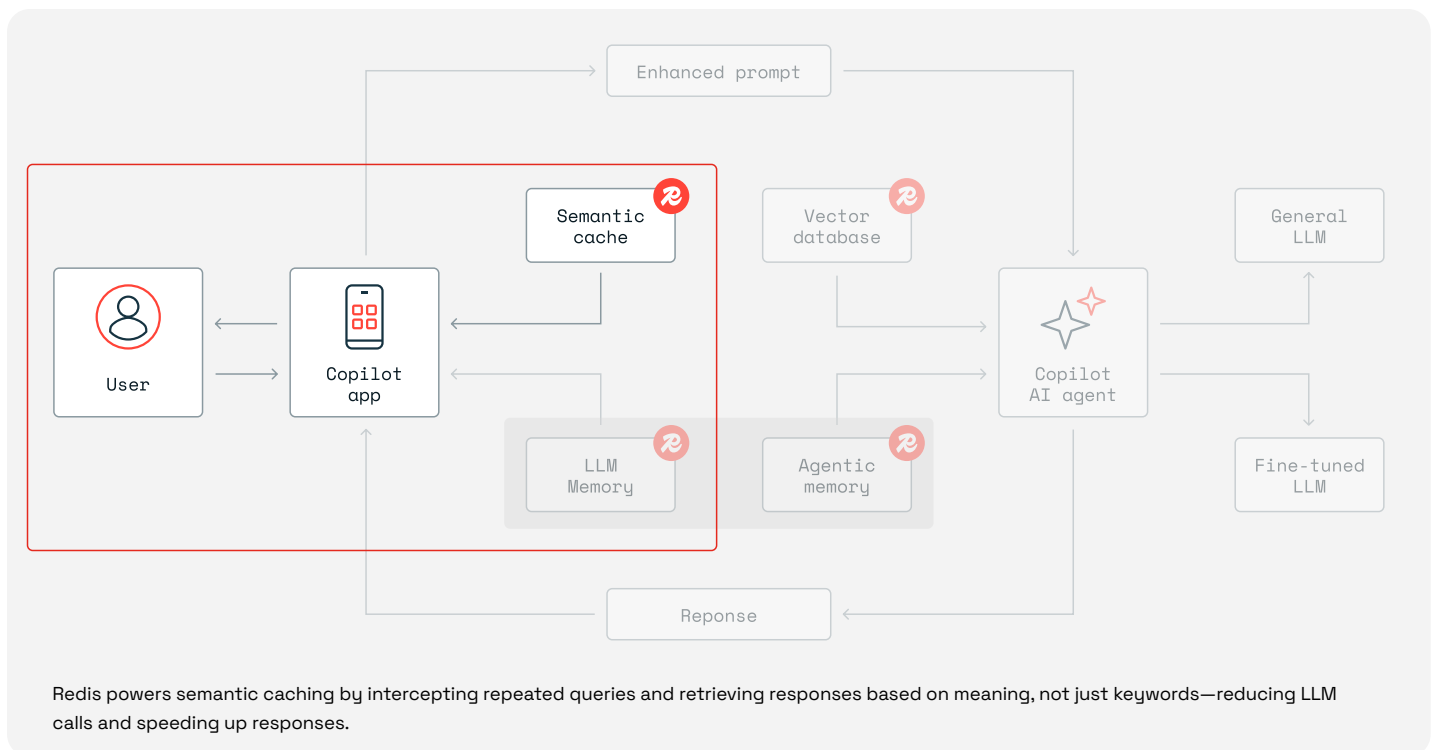
To reduce inference costs and accelerate response times, Redis enables semantic caching—a smarter approach to reusing LLM outputs. The idea is simple but powerful: not every prompt needs to hit the LLM. Instead, you can cache previous LLM responses using semantic fingerprints, or vector embeddings, which capture the meaning behind each prompt. These embeddings are stored in Redis, allowing for rapid lookups based on meaning rather than exact phrasing.

When a new prompt comes in, Redis performs a cosine similarity search to find any semantically similar past prompts. If a close enough match is found, the cached response is returned instantly—no LLM call required. If there isn't a match within the defined similarity threshold, the app gracefully falls back to the LLM, retrieves the response, and then updates the cache with the new query-response pair.

This approach drives both speed and cost efficiency. In Redis Copilot, for example, semantic caching has helped reduce LLM costs by more than 30%, while also making responses 15x faster on average. By caching answers to common or repeated questions—especially in developer tooling use cases—Redis ensures a snappy experience for users while keeping token usage in check.

Redis makes this possible through its ultra-low latency vector search, enabling instantaneous matching of incoming prompts to cached ones. It's a seamless and production-ready way to make GenAI apps more efficient, more affordable, and more responsive.

[Watch episode 2 →](#)
Semantic caching



Step 3: Add agentic memory for smarter, adaptive agents

To build more intelligent and adaptive AI agents, you need to give them memory—the ability to recall knowledge across sessions and interactions. Stateless agents fall short when it comes to long-term reasoning, personalization, or complex task completion. That’s where Redis comes in. With its ultra-fast memory storage, Redis enables agentic workflows that remember goals, facts, interactions, and context over time.

Start by defining memory structures for your agent. Set up distinct memory slots to store system prompts, high-level objectives, factual information, and historical interactions with users. This persistent memory helps the agent retain critical context beyond a single prompt.

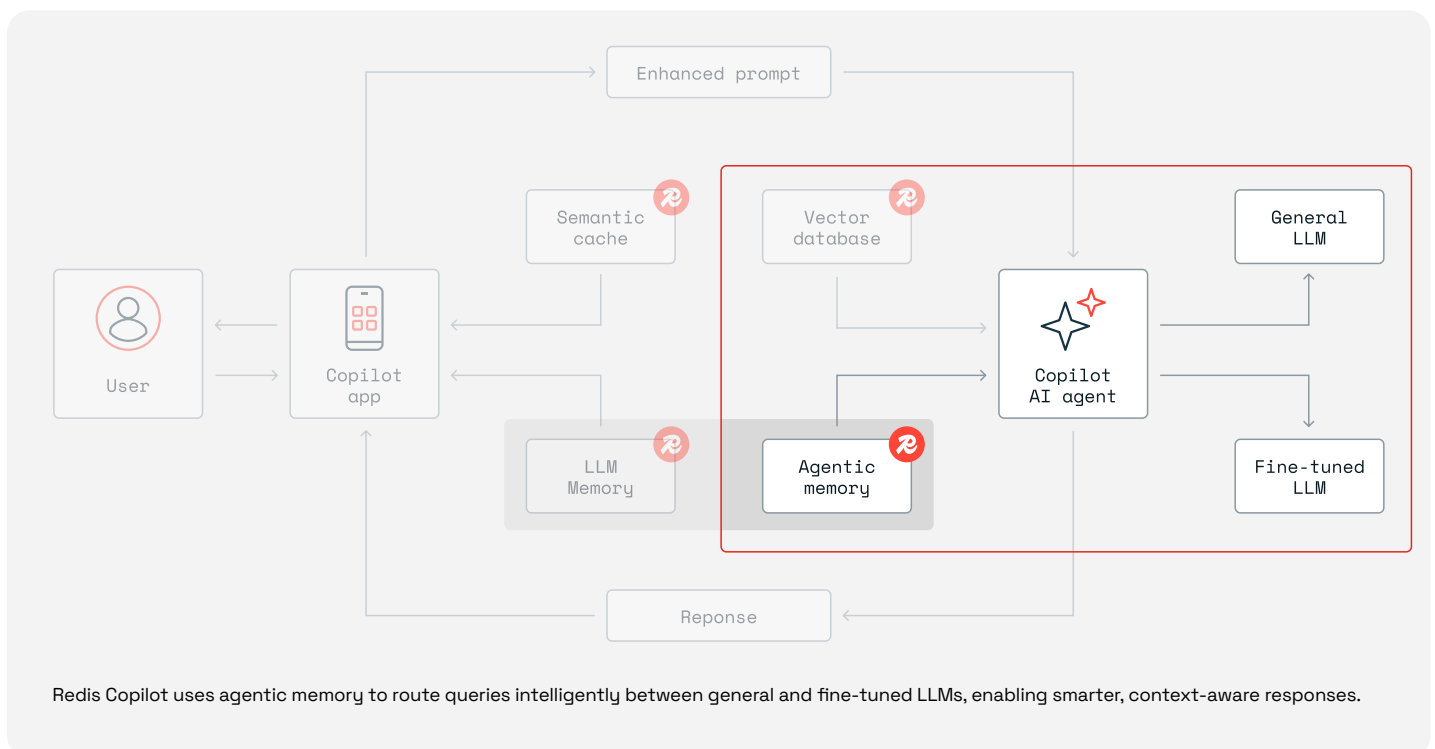
Next, design your agent to use that memory context to route intelligently. In Redis Copilot, for instance, agentic memory powers the decision-making logic that determines whether a query should be answered by a general-purpose LLM or a fine-tuned model specialized in Redis. This enables smarter, more relevant responses.

As the agent interacts with users, it must also be able to evolve. That means implementing memory update strategies that capture new facts or user preferences in real-time. This continuous learning loop makes your agent more useful with each session.

Redis also enables long-term recall for more consistent, personalized reasoning. Agents can reference past conversations or user goals to avoid repetition and provide continuity—just like a human assistant would.

Finally, with Redis’ fast performance and built-in support for semantic routing, agents can leverage their stored knowledge to dynamically choose the right tool for the task—whether that’s a model, a search engine, or an external service. These kinds of compound systems are already being used in production, especially in industries like financial services, where customers expect fast, accurate, and context-aware support.

[Watch episode 3 →](#)
Agentic memory



Step 4: Keep your app safe with rate limiting

In any AI app, especially one exposed to the public, uncontrolled traffic can lead to skyrocketing costs, degraded performance, and even downtime. That's why rate limiting is essential—not just to maintain efficiency, but to ensure your AI stack remains secure and scalable. Redis offers a proven solution through token bucket-based rate limiting with fine-grained control, allowing you to throttle usage dynamically and intelligently.

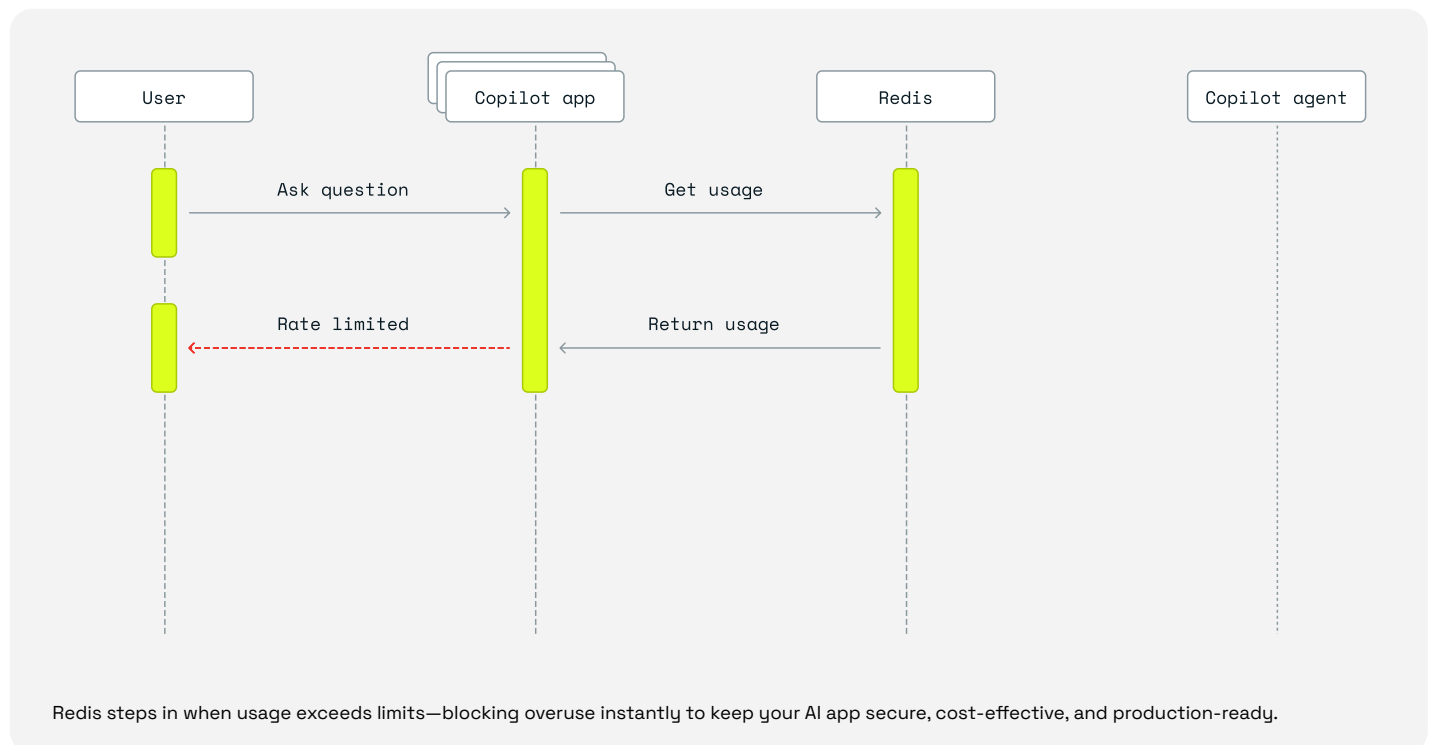
To start, you'll want to set usage limits at the user level. In the Copilot example, Redis tracks how often a user calls the LLM and caps that usage to prevent runaway costs or misuse. These counters are incremented per request and monitored in real-time. Next, implement endpoint-level throttling to prevent abuse at critical points—like your LLM endpoints—where traffic can spike unpredictably, either due to faulty apps or malicious bots.

Redis also makes it easy to define custom rules for different user tiers, enabling differentiated experiences. Premium users can enjoy more generous usage limits, while free-tier users can be constrained to reduce exposure. These rules can be tuned to align cost with value and support monetization strategies.

Crucially, Redis tracks and enforces these limits in real-time. When thresholds are exceeded, Redis blocks requests immediately, sending a clear rate-limit message to the user. This ensures your app remains responsive and your infrastructure protected.

Finally, Redis provides built-in observability tools to trigger alerts and monitor usage patterns, so you can identify anomalies, adapt policies, and confidently scale to production.

[Watch episode 4 →](#)
Rate limiting



Step 5: Speed up ML predictions with Redis feature store

To make accurate predictions, machine learning models need fast, reliable access to the features that power them. Redis acts as a real-time feature store that improves both training and inference by managing features in a single, consistent location. Whether you're detecting fraud in milliseconds or powering a recommendation engine that adapts on the fly, Redis ensures your ML models are fed with the right data, exactly when they need it.

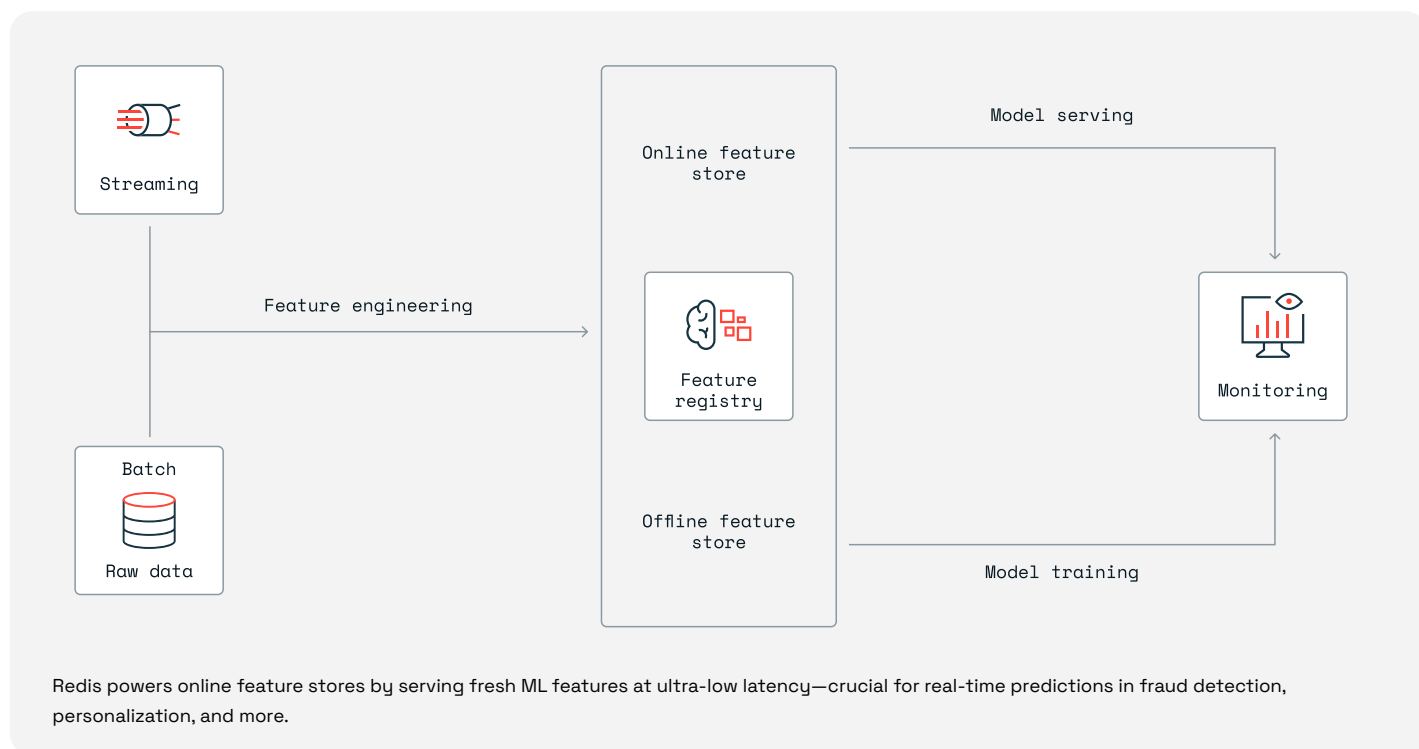
Why does this matter? Because models are only as good as the inputs they receive—and in the world of real-time AI, latency can make or break the user experience. Redis solves this by offering sub-millisecond access to feature data, backed by a high-throughput key-value store and native vector search. This architecture keeps predictions fast, accurate, and scalable even under heavy loads.

To implement a Redis feature store, start by storing precomputed features—such as user scores, metadata, or derived attributes—in Redis for instant access during inference. This is critical for use cases like fraud detection, recommendation systems, or personalized pricing where every millisecond counts. Then, ensure consistency across training and inference by syncing feature values between offline stores and Redis using a feature registry or integrations with tools like Tecton or Feast.

Finally, Redis seamlessly connects to your existing ML stack, whether you're training models in SageMaker or running inference with TensorFlow or PyTorch, thanks to simple SDKs and APIs that plug directly into your pipeline.

[Watch episode 5 →](#)

Feature stores for ML pipelines



Final thoughts & what to build next

From fast vector search to smart caching and real-time pipelines, Redis powers the core infrastructure behind today's most advanced AI apps. Whether you're solving for speed, memory, accuracy, or scale, Redis helps you go from prototype to production with confidence.

Now that you've seen what's possible, what will you build next?



Try creating a chatbot that remembers past conversations.



Launch an autonomous agent that adapts to user goals.



Build a fraud detection system that reacts in real time.

Whatever your next step, Redis is ready.

- [Book a meeting](#) to talk through your use case
- Try [Redis for AI](#)
- Explore LangCache for agent memory
 - [LangCache docs](#)
 - [Short-term memory demo](#)
 - [Long-term memory demo](#)
- Watch more real-time AI demos on the [Redis for AI YouTube channel](#)

Let's build the future of AI—together.

